

STM32 毕设项目 — 软件需求生成模板 v4

工作流：

1. 你在本模板中填写「项目输入」

2. 整份文档发给 Claude（对话）

3. Claude 输出一份**软件需求文档**（.md 文件）

4. 你把该需求文档发给 **Claude Code**，Claude Code 在项目源码中执行开发

你只需要填写下面的「项目输入」部分。框架上下文已内嵌在文档末尾，Claude 会自动参考。

项目输入（每次新题目填写这里）

1. 任务需求【必填】

本项目聚焦现代人群睡眠障碍、焦虑、专注力不足等痛点，结合白噪音的舒缓疗愈特性与智能交互技术，打造兼具实用性与便携性的智能疗愈设备。相比传统白噪音播放器，其解决了功能单一、控制不便、场景适配性弱的问题，可满足卧室助眠、办公专注、学习放松等多场景需求。技术层面，融合音频播放、智能控制、场景化算法等技术，契合物联网与健康家电融合的发展趋势，能系统锻炼本科阶段硬件选型、软件编程、人机交互设计及系统集成能力，方案兼具创新性与实用性，为同类健康智能设备设计提供参考，具备一定市场应用价值。核心功能：内置15+类高品质白噪音（雨声、森林、海浪、溪流、白噪、咖啡馆等），支持自定义混音（如“雨声+鸟鸣”组合）；多模式定时关闭（5/10/15/30/60分钟可选，适配助眠、专注场景）；亮度可调柔光夜灯（暖光2700K，搭配白噪音增强助眠效果）；双重控制方式（本地触控面板+手机APP远程控制，支持音量调节、曲目切换、定时设置）；场景化预设模式（助眠、专注、冥想、儿童哄睡），一键切换对应白噪音与亮度组合。关键指标：夜灯亮度0-100%可调，功耗≤3W；支持蓝牙5.0连接，传输距离≥10米；续航≥8小时（充电式），待机功耗≤0.5W；运行噪音≤30dB；支持TF卡扩展存储。

2. APP 通信变量布局【选填，有 APP 通信则必填】

基于框架的 appcom 模块（remoteVar_TX[16] / remoteVar_RX[16]），规划本项目的变量分配。

不填则由 Claude 根据功能自动分配。

TX（STM32 → APP）

索引	有效宽度	变量含义	数据类型
[0]	4B		
[1]	4B		
[2]	1B		
[3]	1B		
[4]	4B		
[5]	4B		

索引	有效宽度	变量含义	数据类型
[6]	1B		
[7]	1B		

RX (APP → STM32)

索引	有效宽度	变量含义	数据类型
[0]	4B		
[1]	4B		
[2]	1B		
[3]	1B		
[4]	4B		
[5]	4B		
[6]	1B		
[7]	1B		

3. 补充说明【选填】

MP3模块的SD卡中已储存15种背景音乐。
串口屏模块页面，通讯协议如图：

白噪音疗愈音箱



一、串口屏→STM32（固定16字节帧）

帧结构：[0]0x55 包头1→[1]0xAA 包头2→[2]控制字段→[3]~[12]数据内容(ASCII，不足补0x00)→[13]0x00 CRC占位→[14]0x00 CNT占位→[15]0xFF 包尾

控制字段分配：0x05=音频控制，0x06=定时控制，0x07=灯光控制，0x08=模式控制

按钮命令：

- b0 上一曲：55 AA 05 50 52 45 56 00 00 00 00 00 00 00 FF（数据"PREV"）
- b1 播放/暂停：55 AA 05 50 4C 41 59 00 00 00 00 00 00 00 FF（数据"PLAY"）
- b2 下一曲：55 AA 05 4E 45 58 54 00 00 00 00 00 00 00 FF（数据"NEXT"）
- b6 音量+：55 AA 05 56 4F 4C 2B 00 00 00 00 00 00 00 FF（数据"VOL+"）
- b7 音量-：55 AA 05 56 4F 4C 2D 00 00 00 00 00 00 00 FF（数据"VOL-"）
- b3 定时：55 AA 06 54 49 4D 45 00 00 00 00 00 00 00 FF（数据"TIME"）
- b4 灯光：55 AA 07 4C 49 47 48 54 00 00 00 00 00 00 FF（数据"LIGHT"）
- b5 模式：55 AA 08 4D 4F 44 45 00 00 00 00 00 00 00 FF（数据"MODE"）

二、STM32→串口屏（TJC指令协议）

格式：控件名.txt="内容" + 0xFF 0xFF 0xFF，编码GB2312

控件对应：

- t3 = 当前曲目，例：t3.txt="雨声" → 74 33 2E 74 78 74 3D 22 D3 EA C9 F9 22 FF FF FF
- t4 = 定时状态，例：t4.txt="30m" → 74 34 2E 74 78 74 3D 22 33 30 6D 22 FF FF FF
- t5 = 灯光状态，例：t5.txt="暖白" → 74 35 2E 74 78 74 3D 22 C5 AF B0 D7 22 FF FF FF
- t7 = 模式状态，例：t7.txt="助眠" → 74 37 2E 74 78 74 3D 22 D6 FA C3 DF 22 FF FF FF
- t10 = 当前音量，例：t10.txt="80" → 74 31 30 2E 74 78 74 3D 22 38 30 22 FF FF FF

== 以下为框架上下文（不要修改，Claude 自动参考） ==

<stm32_framework_context>

框架概述

基于 STM32F103C8T6，HAL库 + FreeRTOS CMSIS_V2，分层架构。

开发边界：

- **主要工作：** 编写 task 层（应用层）代码，调用 driver 层已有 API
- **允许修改 driver 层的情况：** 当某个功能仅靠现有 driver API 无法实现，但通过修改/扩展 driver 层可以实现时，允许修改 driver 层。修改时必须遵守 driver 层的接口规范和命名规范，并在功能实现文档中明确标注修改了哪个 driver 的哪些内容
- **不可修改：** stdlib 层、HAL 层

分层架构与调用规则

task层（userApp）	← 主要工作区，开发应用逻辑
func层（userFunc）	← 可选，跨driver逻辑整合（如appcom已实现）
driver层（userDriver）	← 已完成，现有API不足时允许修改/扩展
stdlib层（userLib）	← 已完成，不可修改
HAL层（Core/）	← CubemX生成，不可修改

task层可调用：stdlib层、driver层、func层。禁止直接调用 HAL。

文件结构

MDK-ARM/	
├─ userLib/	# stdlib层（已完成）
├─ userDriver/	# driver层（按项目提供）
├─ userFunc/	# func层（appcom等已实现）
└─ userApp/	# task层（开发工作区）
├─ task_system.c/.h	← 系统任务，不修改
└─ task_user1.c/.h	← 应用任务（主要开发文件）

driver 模块自动发现： 项目中 userDriver/ 目录下保留的 .c/.h 文件即为本项目全部可用的 driver 模块（项目负责人已删除无关模块）。开发前先浏览 userDriver/ 目录，逐个阅读每个 driver 的 .h 文件中「向上提供」部分，了解可用的结构体和 API，再开始 task 层设计。

driver 层接口规范

每个 driver 模块提供：

1. 公有信息结构体 xxxInfo （或 xxxInfo[CH_COUNT] ）， task 层通过它读取状态
2. DRIVER_XXX_Init() — 初始化（在 task 的 Init 中调用一次）
3. DRIVER_XXX_Update() — 部分 driver 需要周期调用以更新状态
4. 其他功能函数 DRIVER_XXX_DoSomething(param)

task 层开发规范

框架有两个 FreeRTOS 任务：

- **task_system**: 基础周期 **10ms**, 负责 stdlib 状态机维护和 appcom 通信更新。不需要修改此文件。
- **task_user1**: 基础周期 **2ms**, 应用业务逻辑的主要开发文件, 通过此文件完成项目功能。

每个任务文件必须包含:

1. 公有信息结构体 `xxxTaskInfo_t` + `extern xxxTaskInfo_t xxxTaskInfo`
2. 初始化函数 `xxxTaskInit(void)`
3. 更新函数 `xxxTaskUpdata(void *argument)` — FreeRTOS 任务入口

标准模板:

```
// task_user1.h
typedef struct {
    uint32_t taskCnt;
    // 任务相关状态数据...
} user1TaskInfo_t;
extern user1TaskInfo_t user1TaskInfo;
void user1TaskInit(void);
void user1TaskUpdata(void *argument);

// task_user1.c
user1TaskInfo_t user1TaskInfo;

void user1TaskInit(void) {
    DRIVER_XXX_Init();
    // 初始化任务内部状态
}

void user1TaskUpdata(void *argument) {
    user1TaskInit();
    for(;;) {
        user1TaskInfo.taskCnt++;
        DRIVER_XXX_Update(); // 需要周期更新的driver

        // 通过 taskCnt 取余获得更长的子周期 (基础周期 2ms)
        if(user1TaskInfo.taskCnt % 5 == 0) { // 10ms
            // 10ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 50 == 0) { // 100ms
            // 100ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 250 == 0) { // 500ms
            // 500ms周期逻辑
        }
        if(user1TaskInfo.taskCnt % 500 == 0) { // 1s
            // 1s周期逻辑
        }

        osDelay(2); // 基础调度周期 2ms
    }
}
```

APP 通信模块（func_appcom，已实现）

通信链路：STM32 → USART1 → ESP32 → BLE → APP（双向）

task 层只操作 remoteInfo 结构体，FUNC_APPCOM_UPDATA() 由 task_system 自动调用。

变量池：remoteVar_TX[16] / remoteVar_RX[16]，每个元素为 remoteVar_t（4字节联合体）：

```
typedef union {
    uint8_t raw[4];
    struct __attribute__((packed)) {
        uint32_t var_uint32;
        int      var_int32;
        float     var_float;
    };
} remoteVar_t;
```

索引与帧槽位映射（TX/RX 各16个）：

- [0][1][4][5][8][9][12][13] → 4字节槽位（float/uint32/int32 均可）
- [2][3][6][7][10][11][14][15] → 1字节槽位（仅 var_uint32 低8位有效）

使用方式：

```
// 发送
remoteInfo.remoteVar_TX[0].var_float = currentTemp;
// 接收
float target = remoteInfo.remoteVar_RX[0].var_float;
uint8_t appKey = remoteInfo.key1;
```

编码约束

约束项	规则
内存分配	禁止 malloc，全部静态分配
注释语言	中文
私有定义	写在 .c 文件中
公有定义	写在 .h 文件中
命名风格	task层：小驼峰（user1TaskInit）；结构体：xxxTaskInfo_t；宏：MODULE_CONSTANT
额外功能	不实现需求未提及的功能

</stm32_framework_context>

==== Claude 输出规则（Claude 读到此处后按以下规则输出）====

收到本模板后，Claude 输出一份 **软件需求文档**（.md 文件），该文档将被直接发送给 Claude Code 作为执行指令。

文档必须包含以下结构：

输出文档结构

一、项目概述

简要说明项目做什么、核心功能有哪些。

二、开发前置步骤

指示 Claude Code：

1. 浏览 `MDK-ARM/userDriver/` 目录，阅读每个 driver 的 `.h` 文件中「向上提供」部分
2. 汇总所有可用的结构体和 API
3. 以此为基础进行后续开发

三、task 层设计

1. **文件规划** — 需要哪些 task 文件，各自职责（默认只用 task_user1）
2. **公有结构体设计** — user1TaskInfo_t 包含哪些字段，逐一说明用途
3. **Init 函数设计** — 调用哪些 DRIVER_XXX_Init()，初始化哪些状态
4. **Udata 循环设计** — 各功能的执行周期（基于 2ms 基础周期的 taskCnt 取余）、状态机设计、逻辑流程

四、APP 通信变量布局（如有）

列出 remoteVar_TX/RX 的完整分配表，说明每个索引对应的变量含义和数据类型。

五、功能详细需求

逐个功能模块详细描述：

- 功能描述
- 实现逻辑（状态机、算法、判断条件等）
- 涉及的 driver API 调用
- 关键参数和阈值

六、driver 层修改需求（如需要）

如果根据需求分析，现有 driver API 不足以实现某些功能，在此列出：

- 需要修改/新增的 driver 模块
- 修改内容和原因

- 必须遵守 driver 层的接口规范和命名规范

七、输出物要求

指示 Claude Code 完成开发后，必须输出：

1. 代码文件

- task_user1.c / task_user1.h（以及其他需要的 task 文件）
- 如果修改了 driver 层，输出修改后的 driver 文件并用注释标注修改点

2. 功能实现文档 (implementation_report.md)，包含：

(a) 功能实现状态清单

序号	功能描述	状态	说明
1		✅ / ⚠️ / ❌	

状态分三级：✅ 已实现、⚠️ 部分实现（说明缺什么）、❌ 未实现（说明原因）

(b) driver 层修改记录（如有修改）

文件	修改类型	修改内容	原因
----	------	------	----

(c) 已实现功能的测试步骤

对每个已实现的功能，给出具体的测试方法和预期结果，让开发者可以逐项验证。

输出约束

- 需求文档中引用框架规范时，直接写明规则（如命名风格、分层约束），不要让 Claude Code 再去找本模板
- 不要在需求文档中生成 CubeMX 配置
- 不要指示 Claude Code 修改 task_system、stdlib 层、HAL 层
- 需求文档应当自包含——Claude Code 拿到它 + 项目源码就能完成全部开发，不需要其他文档